ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

Học sâu và ứng dụng trong thị giác máy tính (CO5085)

Học kỳ 261 (2026–2027)

Bài tập lớn A1: CNN vs Transformer trên tập ảnh lớn

Mốc A1.2 – Báo cáo so sánh cuối

Giảng viên: Lê Thành Sách, Nguyễn Quốc Minh

Nhóm: Hummingbird

Họ và tên	**MSSV**
Nguyễn Lê Hoàng Hưng	2570214
Nguyễn Thị Linh Trang	2570344

TP. Hồ Chí Minh, ngày 09 tháng 11 năm 2026

Tóm tắt nội dung

Bài so sánh ResNet-50 (CNN) và DeiT-Small (Transformer), hai mô hình cùng quy mô khoảng 22–24 triệu tham số, trên một subset có kiểm soát của Food-101 (101 lớp; 10.100 ảnh train, 5.050 ảnh test), mỗi mô hình ở bốn chế độ: from-scratch, freeze backbone, freeze một nửa và full finetune. Kết quả tốt nhất là DeiT-S full finetune với 76,20% top-1, so với 60,44% của ResNet-50. Pretrain ImageNet là yếu tố quyết định: from-scratch chỉ đạt 17,8% (ResNet-50) và 11,0% (DeiT-S). Với DeiT-S, freeze một nửa backbone giữ được 75,39% top-1 (kém full finetune 0,8 điểm) với một nửa số tham số được train, thời gian train ít hơn 34% và bộ nhớ GPU bằng khoảng một nửa. Do giới hạn thời gian GPU, quy mô dữ liệu và số epoch nhỏ hơn protocol A1.1; các thay đổi được nêu rõ ở Mục 4.3.

Mục lục

1 Giới thiệu **3**

2 Dataset và EDA **3**

2.1 Food-101 . . . 3

2.2 Split . . . 3

2.3 Phân tích dữ liệu . . . 3

3 DataLoader và Augmentation **4**

4 Protocol thí nghiệm **5**

4.1 Mô hình và chế độ huấn luyện . . . 5

4.2 Huấn luyện . . . 5

4.3 Thay đổi so với protocol A1.1 . . . 6

5 Kết quả **6**

5.1 Bảng tổng hợp . . . 6

5.2 Learning curve . . . 7

5.3 Accuracy và chi phí . . . 7

5.4 Phân tích lỗi . . . 8

6 Thảo luận **9**

6.1 CNN và ViT khi train from-scratch . . . 9

6.2 Vai trò của pretrain . . . 9

6.3 Khi nào nên freeze . . . 10

6.4 Vì sao DeiT-S vượt ResNet-50 khi có pretrain . . . 10

6.5 Đánh đổi accuracy và chi phí . . . 10

6.6 Hạn chế . . . 10

7 Kết luận **11**

Tái lập kết quả **11**

1 Giới thiệu

CNN mang sẵn *inductive bias* về tính cục bộ và bất biến tịnh tiến, còn Vision Transformer [2] xử lý ảnh như một dãy patch và học quan hệ giữa chúng bằng self-attention, gần như không có giả định nào về cấu trúc ảnh. Hệ quả thường được nhắc tới là ViT cần nhiều dữ liệu hoặc cần pretrain mới đạt được độ chính xác của CNN. Bài này kiểm tra nhận định đó trên Food-101 [1], với một CNN (ResNet-50 [3]) và một Transformer cùng quy mô (DeiT-Small [9]), mỗi mô hình ở bốn chế độ huấn luyện.

Các câu hỏi nghiên cứu (giữ nguyên từ A1.1):

1. Khi train from-scratch, CNN hay ViT tốt hơn? Vì sao?
2. Pretrain giúp mỗi họ kiến trúc bao nhiêu? ViT có cần pretrain nhiều hơn CNN không?
3. Khi nào freeze có lợi và khi nào nên full finetune?
4. Đánh đổi giữa accuracy và chi phí của từng cấu hình là gì?

2 Dataset và EDA

2.1 Food-101

Food-101 [1] gồm 101 lớp món ăn, 750 ảnh train và 250 ảnh test mỗi lớp, ảnh màu có cạnh dài tối đa 512 px. Ảnh lấy từ foodspotting.com, chỉ dùng cho nghiên cứu và học tập; repo không chứa ảnh, chỉ chứa code tải dữ liệu. Split train gốc có nhiễu nhãn do tác giả cố ý giữ lại, còn split test đã được làm sạch thủ công. Food-101 không phải tập con của ImageNet, nên các mô hình pretrained ImageNet không bị rò dữ liệu test.

2.2 Split

Tất cả split được lấy phân tầng theo lớp với seed 42 bằng `scripts/prepare_data.py`; danh sách ảnh nằm trong `A1/splits/` (Bảng 1). Validation dùng để chọn checkpoint (epoch có top-1 validation cao nhất); test chỉ dùng một lần cho mỗi run.

Bảng 1: Cách chia dữ liệu (seed 42, phân tầng theo lớp).

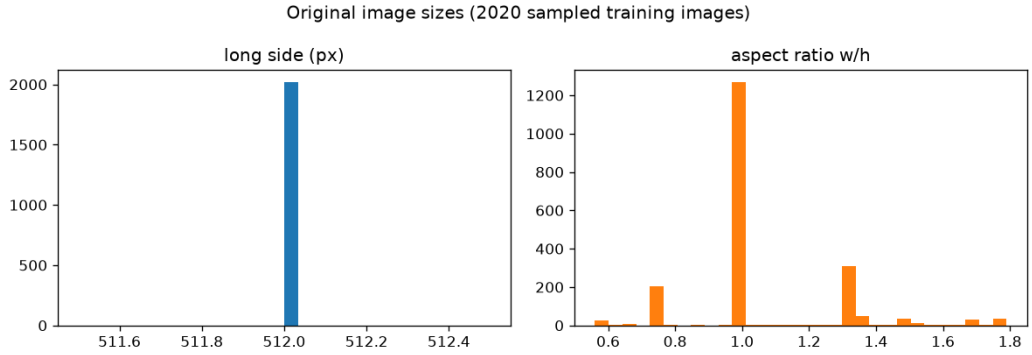
Split	Nguồn	Ảnh/lớp	Tổng
Train	split train gốc	100	10.100
Validation	split train gốc, không giao với Train	20	2.020
Test	split test gốc (đã làm sạch)	50	5.050

2.3 Phân tích dữ liệu

Ba split cân bằng tuyệt đối theo cách lấy mẫu (cùng số ảnh mỗi lớp), nên accuracy và macro-F1 có ý nghĩa tương đương và không cần trọng số lớp. Ảnh gốc có cạnh dài đúng 512 px (trung vị 512 px, đo trên 2020 ảnh train lấy mẫu); 62% là ảnh vuông, phần còn lại có tỉ lệ khung hình khác nhau (Hình 2). Vì vậy resize về cạnh ngắn 256 px rồi center crop 224 px giữ được gần trọn món ăn ở phần lớn ảnh.



Hình 1: Một ảnh train của 12 trong 101 lớp. Ảnh chụp tự nhiên, góc chụp, ánh sáng và bố cục rất khác nhau.



Hình 2: Kích thước ảnh gốc: cạnh dài và tỉ lệ khung hình.

3 DataLoader và Augmentation

Ảnh được resize một lần về cạnh ngắn 256 px (`prepare_data.py`, dùng `Image.draft` để giải mã JPEG ở độ phân giải thấp), vì giải mã ảnh gốc 512 px trên CPU là nút thắt của DataLoader. Chuẩn hóa theo mean/std của ImageNet.

Train:

```
RandomResizedCrop(224, scale=(0.35, 1.0))
RandomHorizontalFlip(p=0.5)
TrivialAugmentWide()
ToTensor(), Normalize(mean, std)
RandomErasing(p=0.25)
```

Validation / Test:

```
Resize(256), CenterCrop(224)
ToTensor(), Normalize(mean, std)
```

DataLoader: batch 64, 8 worker, `pin_memory`, thứ tự xáo trộn cố định bằng generator có seed. Mọi run dùng cùng pipeline, nên giữa hai run chỉ khác kiến trúc hoặc chế độ huấn luyện. TrivialAugment [7] không

cần tinh chỉnh tham số; Random Erasing [12] giảm overfit.

4 Protocol thí nghiệm

4.1 Mô hình và chế độ huấn luyện

Bảng 2: Hai mô hình (checkpoint `timm` [10], pretrained ImageNet-1k). Head luôn được khởi tạo lại cho 101 lớp.

Mô hình	Checkpoint	Tổng tham số	Head
ResNet-50 [3]	`resnet50.a1_in1k` [11]	23.714.981	206.949
DeiT-Small [9]	`deit_small_patch16_224.fb_in1k`	21.704.549	38.885

Bảng 3: Bốn chế độ huấn luyện và số tham số được train.

Chế độ	Phần được train	ResNet-50	DeiT-S
From-scratch	Toàn bộ, khởi tạo ngẫu nhiên	23,71M	21,70M
Freeze backbone	Chỉ head (linear probe)	0,21M	0,04M
Freeze một nửa	ResNet: `layer3`, `layer4`, `fc`; DeiT: `blocks[6:12]`, `norm`, `head`	22,27M	10,69M
Full finetune	Toàn bộ, khởi tạo ImageNet	23,71M	21,70M

Với ResNet, các lớp BatchNorm thuộc phần bị freeze được đặt ở chế độ `eval()` sau mỗi lần `model.train()`, để giữ nguyên running statistics của ImageNet (24 lớp BN ở chế độ freeze một nửa, cả 53 lớp ở chế độ freeze backbone). Phần freeze một nửa của ResNet chiếm 7/16 bottleneck block nhưng chỉ khoảng 6% tham số, vì các tầng đầu ít kênh; phần freeze của DeiT chiếm đúng một nửa tham số vì 12 block có cùng kích thước.

4.2 Huấn luyện

Training loop tự viết (`src/train.py`): forward → cross-entropy với label smoothing 0,1 [8] → `backward()` → `optimizer.step()`. AdamW [5], warm-up tuyến tính 10% số bước đầu rồi cosine [4] về 0, mixed precision fp16 [6]. Bias và tham số LayerNorm/BatchNorm không chịu weight decay.

Bảng 4: Hyperparameter (learning rate và weight decay theo protocol A1.1).

Chế độ	Epoch	LR head	LR backbone	Weight decay
From-scratch	10	10^{-3}	10^{-3}	0,05
Freeze backbone	5	10^{-3}	–	10^{-4}
Freeze một nửa	5	10^{-3}	10^{-4}	0,05
Full finetune	5	10^{-3}	10^{-4}	0,05

Phần cứng. NVIDIA GeForce GTX 1070 (8GB, Pascal), PyTorch 2.8 bản CUDA 12.6. Đo trên máy này: AMP fp16 nhanh hơn fp32 khoảng 29% với ResNet-50 (142 so với 110 ảnh/s) và 19% với DeiT-S (114 so với 96 ảnh/s), đồng thời giảm gần một nửa bộ nhớ, nên mọi run dùng AMP.

4.3 Thay đổi so với protocol A1.1

Protocol A1.1 cần ước tính 12–15 giờ GPU; tại thời điểm chạy chỉ còn khoảng 2 giờ trước hạn nộp. Thiết kế thí nghiệm (2 mô hình $\times$ 4 chế độ, cùng split, augmentation, learning rate, weight decay cho mọi run) được giữ nguyên; chỉ quy mô bị giảm (Bảng 5). Mọi kết luận bên dưới cần đọc trong bối cảnh này: số liệu tuyệt đối thấp hơn mức có thể đạt được, đặc biệt ở hai run from-scratch.

Bảng 5: Những điểm khác protocol A1.1.

	Protocol A1.1	**A1.2 (thực tế)**
Ảnh train / validation / test mỗi lớp	300 / 50 / 250	100 / 20 / 50
Epoch from-scratch / pretrained	50 / 15	10 / 5
Batch	64×2 (accumulation) $= 128$	64
Warm-up	5 epoch (scratch), 1 epoch (còn lại)	10% số bước, mọi run
torchvision	bản CUDA	bản CPU (xem README; chỉ dùng cho transform)

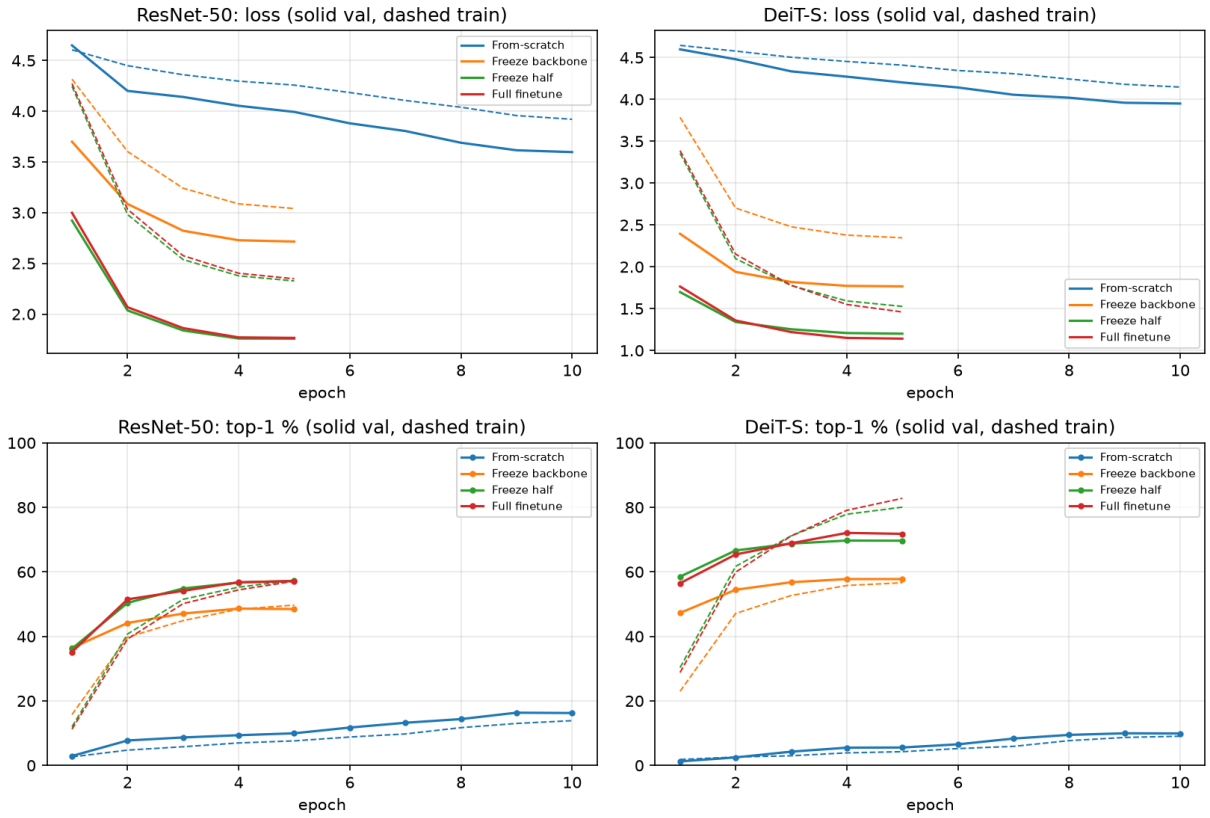
5 Kết quả

5.1 Bảng tổng hợp

Bảng 6: Kết quả trên tập test (5.050 ảnh, checkpoint có top-1 validation cao nhất). Trainable: số tham số được train. s/epoch: trung vị thời gian một epoch train. Thời gian train: tổng mọi epoch, phút. VRAM: bộ nhớ GPU cấp phát tối đa khi train, GB (gồm cả workspace của cuDNN). img/s: tốc độ suy luận, batch 64, AMP. Accuracy và F1 tính bằng %.

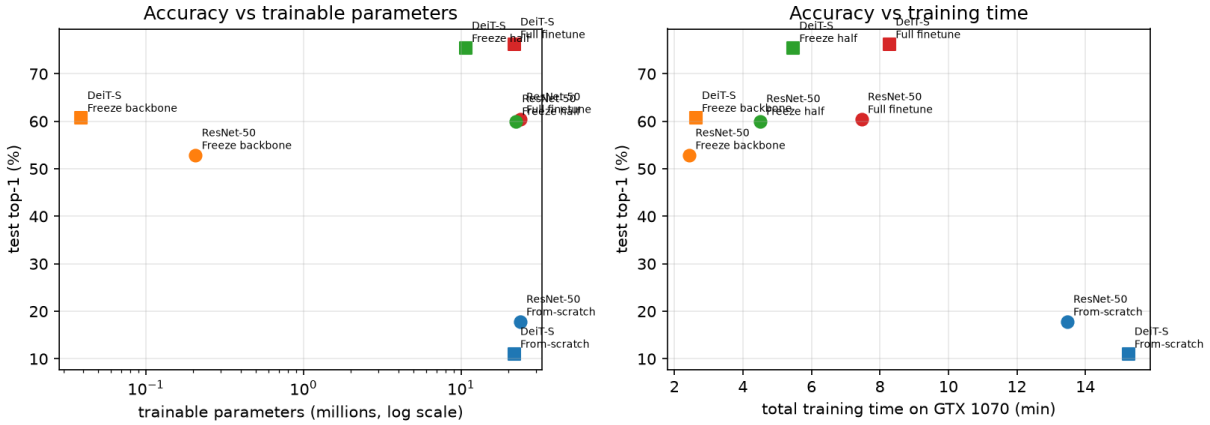
Mô hình	**Chế độ**	**Trainable**	**Epoch**	**Val**	**Top-1**	**Top-5**	**F1**	**s/epoch**	**Phút**	**VRAM**	**img/s**
ResNet-50	From-scratch	23,71M	10	16,34	**17,80**	40,77	14,81	75	13,5	6,5	491
	Freeze backbone	0,21M	5	48,61	**52,85**	80,91	51,00	24	2,4	2,7	479
	Freeze một nửa	22,27M	5	57,23	**60,02**	85,94	58,94	46	4,5	5,7	453
	Full finetune	23,71M	5	57,13	**60,44**	85,82	59,37	78	7,5	5,3	448
DeiT-S	From-scratch	21,70M	10	9,95	**11,01**	30,32	7,99	89	15,3	2,4	437
	Freeze backbone	0,04M	5	57,77	**60,69**	86,46	59,87	27	2,6	0,3	434
	Freeze một nửa	10,69M	5	69,70	**75,39**	93,72	75,10	60	5,5	1,3	412
	Full finetune	21,70M	5	72,08	**76,20**	93,82	75,97	94	8,3	2,4	414

5.2 Learning curve



Hình 3: Loss (hàng trên) và top-1 (hàng dưới) theo epoch. Nét liền: validation; nét đứt: train. Accuracy train được đo trên ảnh đã augmentation, nên không so trực tiếp được với validation.

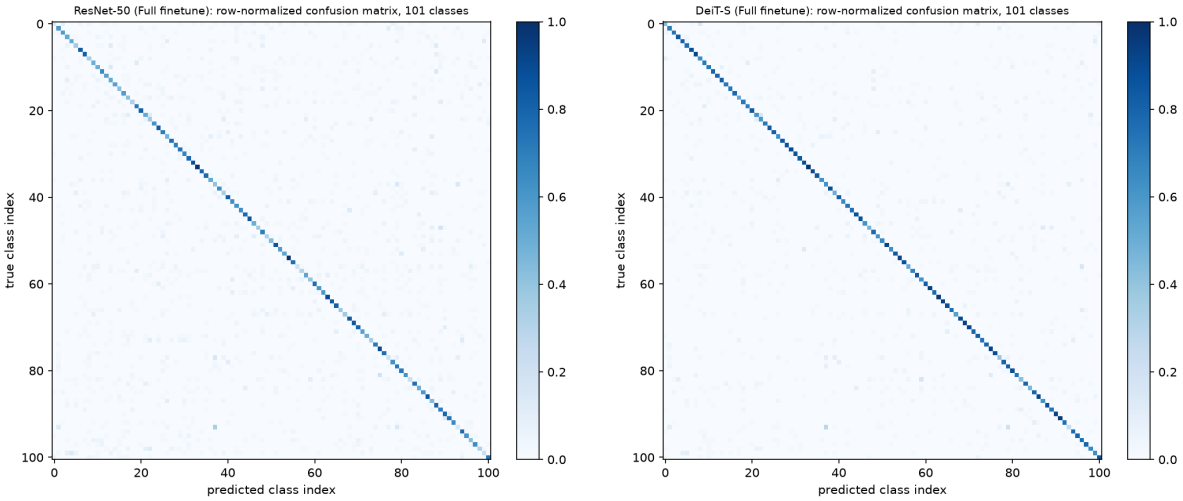
5.3 Accuracy và chi phí



Hình 4: Top-1 trên test theo số tham số được train (trục log) và theo tổng thời gian train. Tròn: ResNet-50; vuông: DeiT-S.

5.4 Phân tích lỗi

Phân tích trên run tốt nhất của mỗi kiến trúc: ResNet-50 Full finetune (60,44%) và DeiT-S Full finetune (76,20%).



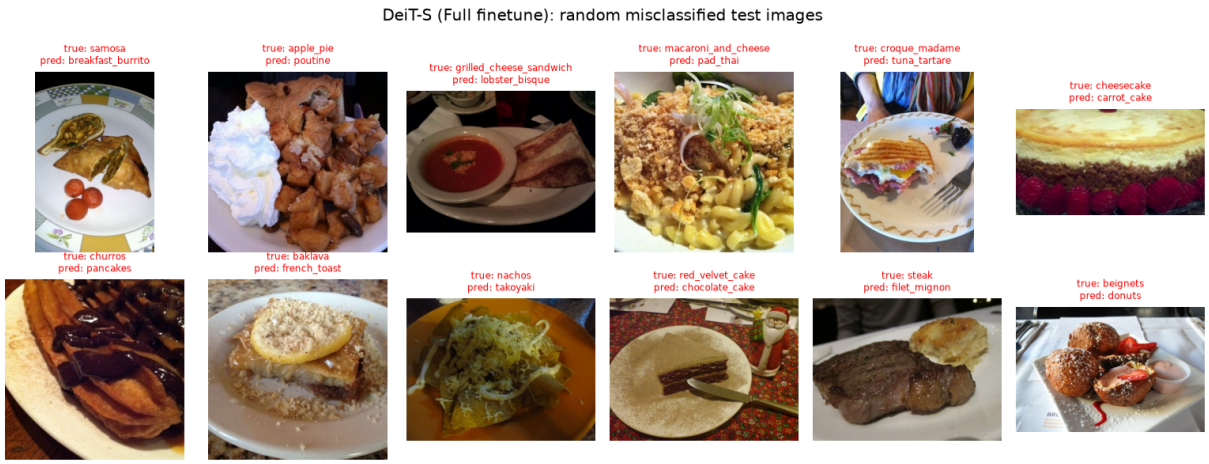
Hình 5: Confusion matrix chuẩn hóa theo hàng (101 lớp, theo thứ tự chữ cái). Đường chéo đậm hơn ở DeiT-S.

Bảng 7: Các cặp lớp bị nhầm nhiều nhất trên test (thật → dự đoán, số ảnh trên 50).

Mô hình	Cặp nhầm nhiều nhất
ResNet-50	steak → filet mignon (17), ramen → pho (10), gnocchi → shrimp and grits (9), pork chop → filet mignon (8), tuna tartare → beef tartare (7), steak → prime rib (7)
DeiT-S	steak → filet mignon (14), steak → prime rib (10), ravioli → lasagna (8), huevos rancheros → tacos (8), chocolate cake → chocolate mousse (8), apple pie → bread pudding (7)

Bảng 8: Năm lớp khó nhất và dễ nhất (accuracy trên 50 ảnh test mỗi lớp).

Mô hình		Lớp
ResNet-50	Khó nhất	apple pie (14%), steak (14%), pork chop (20%), huevos rancheros (20%), tuna tartare (22%)
	Dễ nhất	hot and sour soup (100%), edamame (100%), macarons (90%), pho (90%), dumplings (88%)
DeiT-S	Khó nhất	steak (24%), pork chop (40%), ravioli (42%), apple pie (44%), risotto (46%)
	Dễ nhất	edamame (98%), macarons (96%), spaghetti carbonara (96%), pho (94%), oysters (94%)



Hình 6: 12 ảnh test bị DeiT-S dự đoán sai, chọn ngẫu nhiên.

- **Lỗi tập trung ở các món giống nhau.** Cặp nhầm nhiều nhất của cả hai mô hình là steak → filet mignon; các cặp khác cũng là món cùng nguyên liệu hoặc cùng cách trình bày (ramen và pho, ravioli và lasagna, tuna tartare và beef tartare). Các lớp này chỉ khác nhau ở chi tiết nhỏ hoặc ở cách gọi tên, như Food-101 được thiết kế để thử phân loại chi tiết.
- **Hai mô hình khó và dễ ở cùng các lớp.** Steak, apple pie và pork chop nằm trong nhóm khó nhất của cả hai; edamame, pho, macarons và hot and sour soup nằm trong nhóm dễ nhất, vì có màu sắc và hình dạng đặc trưng. DeiT-S nâng các lớp khó nhiều nhất (apple pie và pork chop từ 14–20% lên 36%), nhưng steak vẫn dưới 20% ở cả hai.
- **Một phần lỗi do chính dữ liệu.** Trong Hình 6, một số ảnh sai có nhiều món trên cùng đĩa, chụp gần một phần món ăn hoặc thiếu sáng; một số nhãn khó xác định ngay cả với người xem. Mỗi lớp chỉ có 50 ảnh test, nên accuracy theo lớp dao động khoảng ± 7 điểm và chỉ nên đọc như xu hướng.

6 Thảo luận

6.1 CNN và ViT khi train from-scratch

ResNet-50 đạt 17,80% và DeiT-S đạt 11,01%, nên ResNet-50 tốt hơn khi không có pretrain, và cách biệt 6,8 điểm lớn hơn nhiều so với sai số đo. ResNet-50 có sẵn inductive bias cục bộ và bất biến tịnh tiến, nên học được đặc trưng dùng được (cạnh, màu, kết cấu) ngay từ ít dữ liệu; DeiT-S phải tự học cả cách các patch liên hệ với nhau chỉ từ 10.100 ảnh. Cả hai còn rất xa các run pretrained và validation vẫn đang tăng ở epoch cuối (ResNet-50: 16,2%, DeiT-S: 9,9%, epoch tốt nhất là 9 và 9), tức là 10 epoch trên 10.100 ảnh chưa đủ để hội tụ. Kết quả này chỉ nói về giai đoạn đầu của quá trình học, không phải mức trần của mỗi kiến trúc.

6.2 Vai trò của pretrain

Pretrain ImageNet là yếu tố lớn nhất trong toàn bộ thí nghiệm: full finetune hơn from-scratch 42,6 điểm với ResNet-50 và 65,2 điểm với DeiT-S, trong khi chỉ train 5 epoch thay vì 10. Mức tăng của DeiT-S lớn hơn ResNet-50 khoảng 22,6 điểm: không có pretrain thì DeiT-S kém ResNet-50, có pretrain thì vượt ở mọi chế độ. Điều này khớp với nhận định của ViT [2] và DeiT [9] rằng Transformer cần pretrain trên dữ liệu

lớn nhiều hơn CNN. Ngay cả linear probe, chỉ train 206,949 (ResNet-50) và 38,885 (DeiT-S) tham số trong khoảng 2,5 phút, cũng vượt from-scratch hơn 35 điểm.

6.3 Khi nào nên freeze

- **DeiT-S:** freeze backbone 60,69%, freeze một nửa 75,39%, full finetune 76,20%. Mở khóa nửa sau tăng 14,7 điểm; mở thêm nửa đầu chỉ tăng 0,8 điểm, nằm trong sai số đo (khoảng $\pm 0{,}6$ điểm với 5.050 ảnh test). Freeze một nửa vì thế là điểm cân bằng tốt nhất: train 10,7M thay vì 21,7M tham số, mất 5,5 thay vì 8,3 phút và 1,3 thay vì 2,4 GB. Các block đầu của ViT học đặc trưng chung (cạnh, màu, kết cấu) dùng lại được cho món ăn; các block sau cần thích nghi với 101 lớp mới.
- **ResNet-50:** freeze backbone 52,85%, freeze một nửa 60,02%, full finetune 60,44%. Freeze một nửa và full finetune ngang nhau. Phần bị khóa (`conv1` đến `layer2`) chỉ chứa khoảng 6% tham số nhưng xử lý feature map độ phân giải cao, nên khóa nó giảm thời gian train từ 7,5 xuống 4,5 phút mà không mất accuracy.
- **Freeze toàn bộ backbone** rẻ nhất (khoảng 2,5 phút, DeiT-S chỉ cần 0,3 GB) và hợp khi cần một baseline nhanh hoặc rất ít dữ liệu, nhưng mất 7,6 điểm (ResNet-50) và 15,5 điểm (DeiT-S) so với full finetune: đặc trưng ImageNet không đủ để phân biệt các món ăn gần nhau nếu không được điều chỉnh.

6.4 Vì sao DeiT-S vượt ResNet-50 khi có pretrain

Ở cả ba chế độ pretrained, DeiT-S cao hơn ResNet-50 từ 7,8 đến 15,8 điểm, dù hai checkpoint có top-1 trên ImageNet gần bằng nhau (khoảng 80%). Thí nghiệm này chưa đủ để kết luận nguyên nhân; các giả thuyết hợp lý là: (1) self-attention toàn cục hữu ích với ảnh món ăn, nơi lớp phụ thuộc vào bố cục và kết cấu của cả đĩa thay vì một vật thể cục bộ; (2) checkpoint `resnet50.a1_in1k` được train với công thức rất mạnh và loss nhị phân [11], có thể chuyển giao sang tập mới kém hơn checkpoint ResNet-50 thông thường; (3) với ResNet-50, learning rate backbone 10^{-4} và 5 epoch có thể quá ít (validation vẫn tăng ở epoch cuối). Kiểm tra (2) và (3) cần thêm run với checkpoint torchvision và lịch train dài hơn.

6.5 Đánh đổi accuracy và chi phí

Tốc độ suy luận của hai mô hình gần bằng nhau (khoảng 410–490 ảnh/s trên GTX 1070), nên khác biệt chi phí nằm ở lúc train. DeiT-S dùng ít bộ nhớ GPU hơn ResNet-50 ở mọi chế độ (full finetune: 2,4 so với 5,3 GB), vì ResNet phải lưu các feature map lớn ở độ phân giải cao cho backward, còn DeiT-S chỉ lưu 197 token 384 chiều mỗi tầng; con số của ResNet còn gồm workspace của cuDNN. Trên Hình 4, DeiT-S freeze một nửa nằm gần góc tốt nhất: gần bằng accuracy cao nhất với chi phí thấp hơn full finetune.

6.6 Hạn chế

- Quy mô nhỏ hơn protocol: 100 ảnh train mỗi lớp, 5 hoặc 10 epoch. Hai run from-scratch chưa hội tụ; khoảng cách giữa from-scratch và pretrained ở đây lớn hơn so với khi train đủ lâu.
- Một seed duy nhất; sai số chuẩn của top-1 trên test khoảng 0,6–0,7 điểm, nên các chênh lệch dưới khoảng 2 điểm (ResNet-50 freeze một nửa so với full finetune, DeiT-S freeze một nửa so với full finetune) không có ý nghĩa thống kê.
- Hyperparameter lấy theo protocol, không tinh chỉnh riêng cho từng mô hình.

7 Kết luận

Trên Food-101 với 100 ảnh train mỗi lớp: (1) pretrain ImageNet quan trọng hơn lựa chọn kiến trúc, đóng góp 43–65 điểm top-1; (2) khi có pretrain, DeiT-S vượt ResNet-50 ở mọi chế độ, tốt nhất 76,20% so với 60,44%; (3) không có pretrain thì ngược lại, ResNet-50 tốt hơn DeiT-S (17,80% so với 11,01%): ViT cần pretrain nhiều hơn CNN; (4) freeze một nửa backbone là lựa chọn hiệu quả nhất: accuracy ngang full finetune trong sai số đo, với thời gian train và bộ nhớ thấp hơn rõ rệt.

Tái lập kết quả

Code: https://github.com/hwnghoang/cs_deeplearning/tree/main/A1. Từ thư mục `A1/`: `python scripts/prepare_data.py` (split và resize), `python run_all.py` (8 run, log và kết quả trong `results/`), `python -m src.analyze` (bảng và hình). Môi trường và các bước tải dữ liệu: `A1/README.md`.

Tài liệu

[1] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101 – mining discriminative components with random forests. In *European Conference on Computer Vision (ECCV)*, 2014.

[2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[4] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017.

[5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[6] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018.

[7] Samuel G. Müller and Frank Hutter. TrivialAugment: Tuning-free yet state-of-the-art data augmentation. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[8] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[9] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning (ICML)*, 2021.

[10] Ross Wightman. Pytorch image models. https://github.com/huggingface/pytorch-image-models, 2019.

[11] Ross Wightman, Hugo Touvron, and Hervé Jégou. ResNet strikes back: An improved training procedure in timm. *arXiv preprint arXiv:2110.00476*, 2021.

[12] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. In *AAAI Conference on Artificial Intelligence*, 2020.